

Nome: _____

- As questões que envolvem implementação devem ser resolvidas utilizando a linguagem C/C++.
 - Utilize as folhas de resposta distribuídas junto com a prova para responder às questões.
 - Em cada folha, escreva seu nome completo no canto superior esquerdo. Respostas em folhas sem identificação não serão corrigidas.
 - As respostas podem ser escritas a lápis ou caneta. Certifique-se de que sua escrita está legível.
 - Indique claramente o número da questão que está respondendo e, quando necessário, detalhe os passos utilizados para chegar à solução.
 - Ao final da prova, as folhas de resposta devem ser grampeadas junto à prova.
-

Questão 1 (1,5 pontos) Calcule o custo computacional de tempo no pior caso, em função de n , das funções abaixo, onde n é o tamanho da entrada. Justifique brevemente.

(a)

```
bool funcA(int* arr, int n) {  
    for (int i = 1; i < 10000; i++) {  
        if (i >= n) {  
            break;  
        } else if (arr[i] < arr[i - 1]) {  
            return false;  
        }  
    }  
    return true;  
}
```

(b)

```
bool funcB(int* arr, int n) {  
    for (int i = 1; i < n; i++) {  
        for (int j = 0; j < i; j++) {  
            if (arr[i] == arr[j]) {  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

Questão 2 (2,5 pontos) Considere a implementação de um navegador web (como Chrome). O navegador possui n abas abertas. Em cada aba há uma página atualmente exibida e um histórico de navegação com capacidade para armazenar, no máximo, m páginas.

O navegador deve permitir as seguintes operações:

- visitar uma nova página na aba atual;
- voltar à página anterior;
- avançar para a próxima página;
- obter a página atualmente exibida.

Responda:

- Quais **Tipos Abstratos de Dados (TADs)** estão envolvidos nesse problema?
- Para cada TAD identificado, indique a **estrutura de dados concreta** que pode ser utilizada para implementá-lo e justifique sua escolha.
- Para cada operação abaixo, descreva como ela pode ser implementada e indique sua complexidade no pior caso:
 - visitar uma nova página;
 - voltar à página anterior;
 - avançar para a próxima página;
 - obter a página atual;
- No pior caso, qual é o número máximo de páginas que podem estar armazenadas simultaneamente no sistema, considerando todas as abas? Expresse sua resposta em função de n e m .

Questão 3 (2,0 pontos) A estrutura a seguir representa uma lista implementada com um array circular:

```
class MyList {  
    private:  
        int* data;      // Vetor que armazena os elementos  
        int capacity;   // Capacidade máxima do vetor  
        int size;       // Número atual de elementos  
        int head;       // Índice do primeiro elemento lógico  
  
    public:  
        bool insertAt(int k, int value);  
};
```

Os elementos são armazenados de forma circular no vetor `data`, sendo `head` o índice do primeiro elemento lógico da lista. Responda:

- Implemente o método `insertAt(int k, int value)`, que insere `value` na posição lógica k , onde $0 \leq k \leq \text{size}$. Caso `size == capacity`, retorne `false` sem inserir; caso contrário, realize a inserção e retorne `true`.
- Qual é o custo computacional do método no pior caso, em função de `size`? Justifique.

Questão 4 (2,0 pontos) Considere a seguinte implementação de uma lista duplamente encadeada:

```
struct Node {
    int data;        // Valor armazenado no nó
    Node* next;      // Ponteiro para o próximo nó
    Node* prev;      // Ponteiro para o nó anterior
};

class LinkedList {
private:
    Node* head; // Ponteiro para o primeiro nó da lista
    Node* tail; // Ponteiro para o último nó da lista

public:
    void removeInterval(int a, int b);
};
```

Assuma que a lista está ordenada em ordem crescente.

- (a) Implemente o método `removeInterval(int a, int b)` que modifica a lista atual removendo todos os nós cujos valores estejam no intervalo inclusivo $[a, b]$.
- (b) Qual é o custo computacional do método no pior caso, em função do tamanho n da lista? Justifique.

Questão 5 (2,0 pontos) Considere um array de tamanho n , onde cada elemento está a no máximo k posições de sua posição no array ordenado. Ou seja, cada elemento pode estar até k posições à esquerda ou à direita de sua posição correta. Responda:

- (a) Entre os algoritmos de ordenação estudados (*Bubble Sort*, *Selection Sort* e *Insertion Sort*), qual você escolheria para ordenar esse array de forma mais eficiente? Justifique brevemente.
- (b) Adapte o algoritmo escolhido para explorar essa propriedade do array, considerando a seguinte assinatura: `void nearlySorted(int* arr, int n, int k)`.
- (c) Qual é a complexidade do algoritmo resultante em função de n e k ? Justifique.